

xv6 十个 Lab 算法流程与参考代码详解

面向答辩抽查的本地仓库学习手册

本文档用于复习 2021 版 xv6 Labs 的实验实现。写作重点放在：每个子实验要解决什么问题、为什么采用这种实现路线、算法执行时数据怎样流动，以及答辩中容易被追问的英文变量和操作系统概念。

推荐阅读方式：先看总览表，再按 Lab 顺序看“设计思路”和“详细算法流程”；遇到代码不熟时，先读关键变量解释，再回到对应代码片段。

0 总体阅读方法

本实现的总体设计方法可以概括为四步：先找入口，再找状态，再找触发路径，最后补测试。入口是用户程序、系统调用或设备中断进入的位置；状态是进程结构、页表项、缓冲区或文件描述符中需要保存的信息；触发路径是某个事件发生后会进入哪段内核逻辑；测试则用官方 grader 和现象输出来验证边界情况。

关键变量与英文名：

- Lab（实验单元）：MIT xv6 课程把一个主题拆成的实验模块，每个 Lab 下又有若干子实验。
- branch（分支）：Git 中保存一组代码状态的名字，本项目用 solution-* 分支分别保存各 Lab 的完成代码。
- syscall（system call，系统调用）：用户程序请求内核服务的接口，例如 read、write、fork。
- trap（陷入）：CPU 从用户态进入内核态处理系统调用、异常或中断的过程。
- page table（页表）：把虚拟地址映射到物理地址的数据结构，是虚拟内存的核心。
- lock（锁）：保护共享数据的同步工具，防止多个 CPU 或线程同时修改同一份状态。

0.1 十个 Lab 总览

Lab	分支	主题	核心设计目标
1	solution-util	Unix utilities (Unix 用户态工具)	练习用户程序、pipe（管道）、fork（创建进程）、exec（替换进程映像）
2	solution-syscall	Syscall（系统调用）	把 trace 和 sysinfo 接入系统调用表，理解用户态到内核态的完整链路
3	solution-pgtbl	Page tables（页表）	理解 Sv39（RISC-V 三级页表机制）、PTE（page table entry，页表项）和用户只读共享页
4	solution-traps	Traps（陷入）	理解 trapframe（陷入帧）、backtrace（调用栈回溯）和定时 alarm
5	solution-cow	Copy-on-write fork（写时复制 fork）	fork 时共享物理页，写入时再复制，降低进程创建成本
6	solution-thread	Multithreading（多线程）	理解用户级线程切换、pthread（POSIX 线程）锁和 barrier（屏障同步）
7	solution-net	Networking（网络）	实现 E1000 网卡收发，理解 descriptor ring

			(描述符环) 和 mbuf (网络包缓冲)
8	solution-lock	Lock (锁优化)	用 per-CPU (每 CPU) 空闲链表和 bucket (哈希桶) 降低锁竞争
9	solution-fs	File system (文件系统)	扩展大文件和 symlink (symbolic link, 符号链接)
10	solution-mmap	Mmap (memory map, 内存映射)	用 VMA (virtual memory area, 虚拟内存区域) 实现文件延迟映射

Lab 1 Unix utilities (Unix 用户态工具)

本章的设计思路是从最小用户程序开始，逐步把系统调用组合起来。sleep 只验证参数和一个系统调用；pingpong 加入进程和管道；primes 用递归管道组织多个进程；find 把目录项递归成深度优先搜索；xargs 把标准输入转换为多次 exec 调用。

1.1 sleep (睡眠工具)

设计思路：先写最小闭环：命令行参数经过用户程序解析后，调用内核已有的 sleep 系统调用，最后退出。这个子实验不追求复杂逻辑，而是确认用户程序能够被编译进 xv6 文件系统并成功触发系统调用。

详细算法流程：

1. 检查 argc (argument count, 参数个数) 是否等于 2；如果不是，说明用户没有传入 tick 数，直接向 fd=2 (standard error, 标准错误) 输出用法。
2. 读取 argv[1] (argument vector, 参数数组中的第 1 个实际参数)，用 atoi (ASCII to integer, 字符串转整数) 转换成 ticks。
3. 调用 sleep(ticks)，用户程序通过 usys.S (用户态系统调用汇编入口) 执行 ecalls 进入内核。
4. 内核根据 ticks 计数让进程进入 SLEEPING 状态，时钟中断唤醒后返回用户态。
5. sleep 返回后调用 exit(0)，表示程序正常结束。

关键变量与英文名：

- argc: 命令行参数数量，程序名本身也算一个参数。
- argv: 字符串指针数组，argv[0] 是程序名，argv[1] 才是用户输入的 ticks。
- ticks: xv6 中的时钟滴答数，不是秒数；一次 tick 由定时器中断推进。

```
if(argc != 2){
    fprintf(2, "usage: sleep ticks\n");
    exit(1);
}
sleep(atoi(argv[1]));
exit(0);
```

答辩口径：可以说 sleep 的核心价值是把“用户参数解析、用户态跳板、内核系统调用、进程睡眠和退出”串成最短路径，因此它是后续系统调用实验的入门模板。

1.2 pingpong (父子进程管道通信)

设计思路：pipe 是单向的，所以要完成父进程向子进程发 ping、子进程向父进程回 pong，需要两条管道。实现时重点不是传多少数据，而是关闭无用端口，保证 read 能在正确时机返回。

详细算法流程：

6. 创建 to_child 和 to_parent 两个 int[2] 数组；pipe(array) 后，array[0] 是读端，array[1] 是写端。
7. 调用 fork (复制当前进程，父进程返回子进程 pid，子进程返回 0) 创建子进程。

- 子进程关闭 `to_child[1]` 和 `to_parent[0]`，因为它只需要从父进程读、向父进程写。
- 子进程 `read(to_child[0], &ch, 1)` 等待 1 字节，读到后打印 received ping，再 `write(to_parent[1], &ch, 1)` 回传。
- 父进程关闭 `to_child[0]` 和 `to_parent[1]`，先写入 1 字节，再读取子进程回传，最后打印 received pong。
- 父进程调用 `wait(0)` 回收子进程，避免子进程变成 zombie（僵尸进程）。

关键变量与英文名：

- `to_child`：父进程写、子进程读的管道。
- `to_parent`：子进程写、父进程读的管道。
- `pid`：process id，进程编号；`fork` 返回值用来区分父子分支。
- handshake（握手）：这里指父子进程用 1 字节完成一次确认。

```
int to_child[2];
int to_parent[2];
char ch = 'x';
pipe(to_child);
pipe(to_parent);

int pid = fork();
if(pid == 0){
    close(to_child[1]);
    close(to_parent[0]);
    read(to_child[0], &ch, 1);
    printf("%d: received ping\n", getpid());
    write(to_parent[1], &ch, 1);
    exit(0);
}
write(to_child[1], &ch, 1);
read(to_parent[0], &ch, 1);
printf("%d: received pong\n", getpid());
wait(0);
```

答辩口径：答辩时重点讲 `fork` 的返回值和 `pipe` 的读写端。父进程和子进程执行同一份代码，但通过 `pid` 分流；管道如果不关闭不用的一端，容易让读端一直等待。

1.3 primes（管道版素数筛）

设计思路：设计成递归进程链：每一层进程只负责确定一个 prime（素数）并过滤它的倍数，剩下的数交给下一层。这样算法结构和 Unix 管道思想一致，每个进程只处理局部职责。

详细算法流程：

- 主进程创建第一条 `pipe`，把 2 到 35 逐个写入写端，然后关闭写端表示输入结束。
- 第一层 `relay` 从左侧管道读出第一个整数，这个数一定是当前范围内最小的未过滤数，因此就是 prime。
- 当前层打印 prime `x`，并创建右侧 `pipe`，准备把不能被当前 prime 整除的数传给下一层。
- `fork` 出下一层子进程；子进程关闭右侧写端，递归调用 `relay(pipefd[0])`。
- 当前层父进程继续从 `leftfd` 读取整数 `n`，如果 `n % prime != 0`，就写入右侧管道。
- 当 `leftfd` 读到 EOF（end of file，输入结束）时，关闭 `leftfd` 和右侧写端，让下一层也能读到 EOF 并退出。
- 每层 `wait` 子进程，最后整条进程链自然收束。

关键变量与英文名：

- `relay`：本实现中每一层筛选进程的函数名，表示把过滤后的数据转发给下一层。
- `leftfd`：当前层左侧输入管道的文件描述符。
- `pipefd`：当前层创建的右侧输出管道。
- EOF：读端返回 0，表示所有写端都关闭，后续没有数据。

```
static void relay(int leftfd)
{
    int prime;
    if(read(leftfd, &prime, sizeof(prime)) != sizeof(prime)){
        close(leftfd);
        exit(0);
    }
    printf("prime %d\n", prime);

    int pipefd[2];
    pipe(pipefd);
    if(fork() == 0){
        close(pipefd[1]);
        close(leftfd);
        relay(pipefd[0]);
    }
    close(pipefd[0]);
    while(read(leftfd, &n, sizeof(n)) == sizeof(n)){
        if(n % prime != 0)
            write(pipefd[1], &n, sizeof(n));
    }
}
```

答辩口径：这个实验不是为了写最快的素数算法，而是为了理解“一个 pipe 连接两个进程”的组合方式。每一层都只知道自己的 prime 和左右管道，不需要全局数组。

1.4 find（递归查找文件）

设计思路：把文件系统目录树看成一棵树，采用 DFS（depth-first search，深度优先搜索）。遇到文件就比较文件名，遇到目录就继续进入子目录，递归参数保存当前路径。

详细算法流程：

19. open(path, 0) 打开当前路径，失败则打印错误并返回。
20. fstat(fd, &st) 读取当前路径的类型和元数据，st.type 可以区分 T_FILE（普通文件）和 T_DIR（目录）。
21. 如果当前节点是文件，调用 basename(path) 取最后一级文件名，与 target 比较；相等就打印完整路径。
22. 如果当前节点是目录，先判断路径缓冲区 buf[512] 是否足够容纳 path + '/' + 子文件名。
23. 循环 read(fd, &de, sizeof(de)) 读取 dirent（directory entry，目录项）。
24. 跳过 de.inum == 0 的空目录项，也跳过 . 和 ..，否则递归会回到当前目录或父目录。
25. 把 de.name 拼到 buf 后面，递归调用 find(buf, target)，直到整棵目录树搜索完。

关键变量与英文名：

- struct stat: 文件元数据结构，包含文件类型、大小、inode 编号等。
- struct dirent: 目录项结构，包含 inum 和定长文件名 name。
- basename: 路径最后一级名字，例如 a/b/c 的 basename 是 c。
- inum: inode number，索引节点编号；为 0 表示这个目录项无效。

```
switch(st.type){
case T_FILE:
    if(strcmp(basename(path), target) == 0)
        printf("%s\n", path);
    break;
case T_DIR:
    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';
    while(read(fd, &de, sizeof(de)) == sizeof(de)){
        if(de.inum == 0)
            continue;
        if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
            continue;
        memmove(p, de.name, DIRSIZ);
```

```

    p[DIRSIZ] = 0;
    find(buf, target);
}
}

```

答辩口径：答辩时可按“打开路径、判断类型、文件比较、目录递归、跳过点目录、防止路径过长”这个顺序讲。核心设计是 DFS，而不是简单遍历当前目录。

1.5 xargs（标准输入转参数执行）

设计思路：xargs 的作用是把前一个命令输出的每一行，追加到后一个命令的参数表里。设计上每读一行就 fork/exec 一次，父进程 wait 后继续读下一行。

详细算法流程：

26. 先检查 argc 至少为 2，因为 argv[1] 才是要执行的命令名。
27. readline 从 fd=0（standard input，标准输入）逐字节读取，遇到换行符或 EOF 停止，并补字符串结尾 0。
28. 创建 args[MAXARG] 参数数组，先复制原始命令参数 argv[1]、argv[2] 等。
29. 把当前读到的 line 作为最后一个普通参数追加到 args 中。
30. 把 args[narg] 置为 0；exec 要靠这个空指针判断参数数组结束。
31. fork 后子进程 exec(args[0], args)，父进程 wait，保证每行输入对应一次独立命令执行。
32. 读到空行或 EOF 后退出。

关键变量与英文名：

- MAXARG: xv6 允许传给 exec 的最大参数数量。
- line: 当前从标准输入读到的一行内容。
- exec: 用新程序替换当前进程的用户地址空间，成功后不会返回原代码。
- args[narg] = 0: 参数表结束标记，缺少它会导致 exec 继续读越界内存。

```

while(readline(line, sizeof(line)) >= 0){
    if(line[0] == 0)
        break;
    int narg = 0;
    for(int i = 1; i < argc && narg < MAXARG - 1; i++){
        args[narg++] = argv[i];
    }
    args[narg++] = line;
    args[narg] = 0;

    if(fork() == 0){
        exec(args[0], args);
        exit(1);
    }
    wait(0);
}

```

答辩口径：可以举例说明：echo hello | xargs echo world，前一个 echo 输出 hello，xargs 把 hello 追加到后一个 echo world 后面，最终执行的是 echo world hello。

Lab 2 Syscall（系统调用）

本章的设计思路是完整接入新系统调用。新增 syscall 不能只写 sys_XXX 函数，还要经过编号、用户态声明、usys.pl（生成用户态汇编跳板的脚本）、内核分发表、参数读取和返回值写回。

2.1 trace（系统调用跟踪）

设计思路：trace 要跟踪所有系统调用，因此打印逻辑放在 syscall() 的统一出口，而不是分散到每个 sys_XXX 函数。每个进程保存自己的 tracemask，fork 时复制，exec 后仍然保留。

详细算法流程：

33. 在 `syscall.h` 中分配 `SYS_trace` 编号；编号会被用户态跳板放入 `a7` 寄存器，内核据此找到具体处理函数。
34. 在 `user.h` 声明 `int trace(int)`，在 `usys.pl` 增加 `entry("trace")`，生成用户态系统调用桩函数。
35. 在 `struct proc` 中加入 `tracemask` (trace mask, 跟踪掩码)，用一个整数的不同 bit 表示要跟踪哪些系统调用。
36. `sys_trace` 从用户传入参数中读取 `mask`，只写入 `myproc()->tracemask`，不做打印。
37. `fork` 中执行 `np->tracemask = p->tracemask`，使子进程继承父进程的跟踪设置。
38. `syscall()` 读取 `p->trapframe->a7` 得到系统调用编号 `num`，调用 `syscalls[num]()` 得到返回值并写回 `a0`。
39. 如果 `p->tracemask & (1 << num)` 不为 0，说明当前系统调用被选中，打印 `pid`、`syscallnames[num]` 和返回值。

关键变量与英文名：

- `tracemask`: 跟踪掩码，二进制位为 1 的位置表示要打印对应编号的系统调用。
- `a7`: RISC-V 约定中保存系统调用编号的寄存器。
- `a0`: RISC-V 约定中保存第一个参数和返回值的寄存器。
- `syscallnames`: 系统调用编号到名字的数组，用于把编号打印成 `read`、`write` 等可读名字。

```
num = p->trapframe->a7;
if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
    if((p->tracemask & (1 << num)) != 0)
        printf("%d: syscall %s -> %d\n",
               p->pid, syscallnames[num], p->trapframe->a0);
}

uint64
sys_trace(void)
{
    int mask;
    if(argint(0, &mask) < 0)
        return -1;
    myproc()->tracemask = mask;
    return 0;
}
```

答辩口径：trace 的关键设计是“状态存在 `proc`，打印在 `syscall` 统一出口”。这样新增系统调用也能自动被跟踪，且 `exec` 不会清掉 `proc` 中的 `tracemask`。

2.2 sysinfo（系统信息查询）

设计思路：`sysinfo` 要把内核统计结果返回给用户结构体，所以设计成内核先统计 `freemem` 和 `nproc`，再用 `copyout` 安全写回用户地址。这样避免用户直接访问内核数据。

详细算法流程：

40. 在 `kernel/sysinfo.h` 中使用 `struct sysinfo` 保存 `freemem` 和 `nproc` 两个字段。
41. `sys_sysinfo` 通过 `argaddr` 读取用户传入的结构体地址 `addr`；这个地址属于用户虚拟地址，不能在内核中直接解引用。
42. `freemem` 遍历 `kmem.freelist`（物理页空闲链表），统计空闲页数，再乘以 `PGSIZE` 得到字节数。
43. `nproc` 遍历全局 `proc[NPROC]` 进程表，统计 `state != UNUSED` 的进程数量。
44. 把统计结果写入局部变量 `info`，最后调用 `copyout(p->pagetable, addr, (char *)&info, sizeof(info))`。
45. `copyout` 会按照当前进程页表把用户虚拟地址翻译成物理地址，并检查地址是否合法。

关键变量与英文名：

- `freemem`: free memory, 当前空闲物理内存字节数。
- `nproc`: number of processes, 当前已分配的进程数量。

- copyout: 从内核缓冲区复制数据到用户地址空间的函数。
- kmem.freelist: 空闲物理页链表, kalloc 从这里取页, kfree 把页放回这里。

```
uint64
sys_sysinfo(void)
{
    uint64 addr;
    struct sysinfo info;
    struct proc *p = myproc();

    if(argaddr(0, &addr) < 0)
        return -1;
    info.freemem = freemem();
    info.nproc = nproc();
    if(copyout(p->pagetable, addr, (char *)&info, sizeof(info)) < 0)
        return -1;
    return 0;
}
```

答辩口径: 答辩时强调用户指针不能直接写, 要通过 copyout。freemem 看物理页分配器, nproc 看进程表, 这两个统计正好连接了内存管理和进程管理。

Lab 3 Page tables (页表)

本章的设计思路是把页表从“黑盒地址转换”变成可以观察和修改的结构。ugetpid 利用共享只读页减少系统调用开销; pgaccess 利用硬件访问位统计页面访问情况; vmprint 用递归方式打印 Sv39 页表树。

3.1 ugetpid (用户态快速读取 pid)

设计思路: getpid 原本每次都要陷入内核。ugetpid 的设计是给每个进程分配一页只读共享页, 内核写 pid, 用户态直接读, 减少 trap 成本。

详细算法流程:

46. 在 proc.h 的 struct proc 中加入 struct usyscall *usyscall, 表示该进程的用户共享页。
47. allocproc 分配进程时 kalloc 一页物理内存给 usyscall, 并写入 p->usyscall->pid = p->pid。
48. proc_pagetable 创建用户页表时, 把这页映射到固定虚拟地址 USYSCALL。
49. 映射权限使用 PTE_R | PTE_U, 表示用户态可读但不可写, 避免用户篡改 pid。
50. freeproc 和 proc_freepagetable 负责释放物理页和取消页表映射, 防止内存泄漏。
51. 用户态 ugetpid 直接读 USYSCALL 页中的 pid 字段, 不需要执行 ecalls。

关键变量与英文名:

- USYSCALL: 用户态只读共享页的固定虚拟地址。
- PTE_R: 页表项 readable, 可读权限位。
- PTE_U: 页表项 user, 允许用户态访问。
- mappages: 在页表中建立虚拟地址到物理地址的映射。

```
if((p->usyscall = (struct usyscall *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
p->usyscall->pid = p->pid;

if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)p->usyscall, PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}
```

答辩口径：这个实现体现了一个操作系统设计取舍：频繁读取且不涉及安全修改的数据，可以通过只读共享页减少系统调用开销。

3.2 pgaccess（页面访问检测）

设计思路：pgaccess 的设计是利用硬件维护的 PTE_A 位。用户给出一段连续虚拟页，内核检查每一页的 accessed 位，把结果压缩到一个整数 mask 中返回。

详细算法流程：

52. sys_pgaccess 读取 base、len、maskaddr 三个参数，其中 base 是起始虚拟地址，len 是页数，maskaddr 是用户结果地址。
53. 限制 len 不能超过 32，因为结果用 unsigned int 的 32 个 bit 保存。
54. 循环 i = 0..len-1，调用 walk(p->pagetable, base + i * PGSIZE, 0) 找对应页表项。
55. 如果 pte 不存在或没有 PTE_V，说明地址非法，返回 -1。
56. 如果 *pte & PTE_A 为真，说明该页被访问过，就执行 mask |= (1U << i)。
57. 检查后清除 PTE_A，保证下一次 pgaccess 只统计新的访问。
58. 最后用 copyout 把 mask 写回用户空间。

关键变量与英文名：

- PTE_A: accessed bit，页被硬件访问过后设置的访问位。
- mask: 位图，bit i 表示第 i 个页面是否被访问。
- walk: 沿三级页表查找某个虚拟地址对应的 PTE 指针。
- PGSIZE: page size，页大小，xv6 中为 4096 字节。

```
for(int i = 0; i < len; i++){
    pte_t *pte = walk(p->pagetable, base + i * PGSIZE, 0);
    if(pte == 0 || (*pte & PTE_V) == 0)
        return -1;
    if(*pte & PTE_A){
        mask |= (1U << i);
        *pte &= ~PTE_A;
    }
}
if(copyout(p->pagetable, maskaddr, (char *)&mask, sizeof(mask)) < 0)
    return -1;
```

答辩口径：答辩时要说清楚 pgaccess 不是自己记录访问历史，而是读取硬件页表项中的 PTE_A。清除 PTE_A 是为了让下次检测有新的基准。

3.3 vmprint（打印页表结构）

设计思路：Sv39 页表是三级树结构，因此打印函数采用递归。每一层遍历 512 个 PTE，遇到有效项就打印；如果该项不是叶子页表项，就继续递归下一层。

详细算法流程：

59. vmprint 先打印根页表地址 page table %p。
60. vmprintwalk 接收 pagetable 和 depth，depth 用来控制前面的缩进数量。
61. 每层循环 512 个 PTE，因为一个页表页大小 4096 字节，每个 PTE 8 字节。
62. 如果 PTE_V 没有设置，说明该项无效，直接跳过。
63. 通过 PTE2PA(pte) 把页表项里的物理页号转换成物理地址。
64. 如果 PTE_R、PTE_W、PTE_X 都没有设置，说明它指向下一级页表，继续递归。
65. 如果是叶子项，说明它直接映射到物理页，只打印不再递归。

关键变量与英文名：

- Sv39: RISC-V 64 位下的三级页表格式，虚拟地址用 3 个 9 bit 索引加页内偏移。
- leaf PTE: 叶子页表项，带有 R/W/X 权限，直接映射物理页。

- depth: 递归深度, 用来决定输出前有几个 '..'。
- PTE2PA: 把 PTE 中保存的物理页号还原成物理地址。

```
static void
vmprintwalk(pagetable_t pagetable, int depth)
{
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if((pte & PTE_V) == 0)
            continue;
        uint64 pa = PTE2PA(pte);
        for(int j = 0; j < depth; j++)
            printf(" ..");
        printf("%d: pte %p pa %p\n", i, pte, pa);
        if((pte & (PTE_R | PTE_W | PTE_X)) == 0)
            vmprintwalk((pagetable_t)pa, depth + 1);
    }
}
```

答辩口径: vmprint 的设计思路就是把页表当树遍历。答辩时可以画出根页表、二级页表、一级页表和叶子页的关系。

Lab 4 Traps (陷入)

本章的设计思路是围绕 trapframe 展开: 发生系统调用、异常或定时器中断时, 必须保存用户态现场; 内核处理完后, 再根据 trapframe 恢复用户程序。backtrace 训练栈帧理解, sigalarm 训练用户态现场保存与恢复。

4.1 backtrace (调用栈回溯)

设计思路: backtrace 利用 RISC-V 函数调用约定: 当前栈帧的 fp 附近保存了返回地址 ra 和上一层 fp。只要沿着 fp 链向上走, 就能打印调用路径。

详细算法流程:

66. 在 riscv.h 中实现 r_fp, 使用汇编 mv 从 s0 寄存器读出 frame pointer。
67. backtrace 中先取当前 fp, 再用 PGROUNDUP 和 PGROUNDDOWN 得到当前内核栈页上下边界。
68. 循环条件要求 fp 落在当前栈页内, 并且至少能读取 fp-8 和 fp-16 两个 8 字节位置。
69. 读取 *(uint64 *) (fp - 8) 得到 ra, 也就是当前函数返回到调用者的位置。
70. 打印 ra, 后续可用 addr2line 把地址转换成源代码行。
71. 读取 *(uint64 *) (fp - 16) 得到上一层栈帧的 fp, 继续回溯。
72. 当 fp 超出当前栈页边界时停止, 防止把无关内存当作栈继续读取。

关键变量与英文名:

- fp: frame pointer, 帧指针, RISC-V 中通常保存在 s0 寄存器。
- ra: return address, 返回地址, 函数结束后要跳回的位置。
- PGROUNDUP: 把地址向上取整到页边界, 例如 0x1234 变为 0x2000。
- PGROUNDDOWN: 把地址向下取整到页边界, 例如 0x1234 变为 0x1000。

```
uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x));
    return x;
}

void
backtrace(void)
{
    uint64 fp = r_fp();
    uint64 top = PGROUNDUP(fp);
```

```

uint64 bottom = PGROUNDDOWN(fp);
while(fp >= bottom + 16 && fp < top){
    uint64 ra = *((uint64 *)(fp - 8));
    printf("%p\n", ra);
    fp = *((uint64 *)(fp - 16));
}
}

```

答辩口径：答辩时用一句话概括：backtrace 不是扫描整个栈，而是沿着每个栈帧保存的上一个 fp 指针向上走，所以边界检查非常重要。

4.2 sigalarm / sigreturn（用户级定时回调）

设计思路：sigalarm 的设计是借用 timer interrupt（定时器中断）周期性打断进程，在返回用户态前把 epc 改成 handler 地址。sigreturn 再把保存的 trapframe 恢复回来，让原程序继续执行。

详细算法流程：

73. 在 proc.h 中给进程增加 alarm_interval、alarm_elapsed、alarm_active、alarm_handler 和 alarmframe。
74. sys_sigalarm(interval, handler) 读取两个参数，把间隔和用户 handler 地址写入当前 proc。
75. usertrap 每次处理定时器中断时，如果 which_dev == 2，就说明这是 timer interrupt。
76. 只有 alarm_interval > 0 且 alarm_active == 0 时才累计 alarm_elapsed，防止 handler 执行中再次重入。
77. 当 alarm_elapsed 达到 alarm_interval，先把当前 trapframe 完整复制到 alarmframe。
78. 把 p->trapframe->epc 改成 alarm_handler；usertrapret 返回用户态后，CPU 会从 handler 开始执行。
79. handler 执行完调用 sigreturn，sys_sigreturn 用 alarmframe 覆盖 trapframe，并清除 alarm_active。
80. 返回值恢复为 alarmframe.a0，避免破坏原用户程序对 a0 的使用。

关键变量与英文名：

- trapframe：陷入帧，保存用户态寄存器和返回地址的结构体。
- epc：exception program counter，异常或中断发生时保存的用户程序计数器。
- handler：用户注册的处理函数地址。
- alarm_active：是否正在执行 handler 的标志，用来避免嵌套触发。

```

if(which_dev == 2 && p->alarm_interval > 0 && p->alarm_active == 0){
    p->alarm_elapsed++;
    if(p->alarm_elapsed >= p->alarm_interval){
        p->alarm_elapsed = 0;
        p->alarm_active = 1;
        memmove(&p->alarmframe, p->trapframe, sizeof(*p->trapframe));
        p->trapframe->epc = p->alarm_handler;
    }
}

uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    uint64 a0 = p->alarmframe.a0;
    memmove(p->trapframe, &p->alarmframe, sizeof(*p->trapframe));
    p->alarm_active = 0;
    return a0;
}

```

答辩口径：可以说 alarm 的本质是“修改返回用户态的位置”。内核没有直接调用用户函数，而是改 trapframe->epc，让 sret 后自然跳到用户 handler。

Lab 5 Copy-on-write fork (写时复制 fork)

本章的设计思路是延迟复制。普通 fork 会复制父进程全部用户页，成本高；COW 让父子进程先共享同一物理页，并把页表改成只读。只有某一方真正写入时，才在 page fault（缺页异常）里分配新页并复制内容。

5.1 物理页引用计数

设计思路：共享物理页后，一个页可能被多个进程页表引用，所以 kfree 不能看到释放请求就立刻回收。需要给每个物理页维护 ref count，只有计数降到 0 才放回 freelist。

详细算法流程：

81. 在 kalloc.c 中新增 refs.count[] 数组，数组下标由物理地址 pa 转换而来。
82. kinit 初始化 refs.lock, freerange 初始释放物理页时先把引用计数设为 1，再调用 kfree 归还。
83. kalloc 从 kmem.freelist 取到页后，把该页引用计数设为 1。
84. kaddrref 用于 COW 共享时给物理页引用计数加 1。
85. kfree 先把引用计数减 1，如果减完仍大于 0，说明还有其他页表引用该页，不真正释放。
86. 只有引用计数变成 0，才 memset 填充调试值并挂回 kmem.freelist。

关键变量与英文名：

- COW: copy-on-write, 写时复制，读时共享、写时复制。
- ref count: reference count, 引用计数，记录物理页被多少个页表引用。
- pa: physical address, 物理地址。
- freelist: 空闲链表，保存可重新分配的物理页。

```
void
kfree(void *pa)
{
    acquire(&refs.lock);
    refs.count[paindex((uint64)pa)]--;
    if(refs.count[paindex((uint64)pa)] > 0){
        release(&refs.lock);
        return;
    }
    release(&refs.lock);
    memset(pa, 1, PGSIZE);
    r = (struct run*)pa;
    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

答辩口径：答辩时强调：COW 的正确性依赖引用计数。没有引用计数，父进程释放页面时可能把子进程仍在使用的物理页回收。

5.2 fork 时建立 COW 映射

设计思路：fork 阶段不复制物理页，只复制页表映射。原来可写的页要去掉 PTE_W，并打上 PTE_C 标记，表示未来写入时需要复制。

详细算法流程：

87. uvmcopy 遍历父进程地址空间中每一页，walk 找到父页表项。
88. 检查 PTE_V，确认该页有效。
89. 如果父页表项有 PTE_W，说明原来是可写页，就把 PTE_W 清掉并设置 PTE_C。
90. 提取物理地址 pa 和权限 flags，让子进程页表映射到同一物理页。
91. 调用 kaddrref(pa)，因为父子页表现在共同引用这一页。
92. mappages(new, i, PGSIZE, pa, flags) 给子进程建立同权限映射。

93. 最后 `sfence_vma` 刷新 TLB，避免 CPU 继续使用旧的可写权限缓存。

关键变量与英文名：

- `PTE_W`: writable, 可写权限位。
- `PTE_C`: 本实现新增的 copy-on-write 标记位。
- `flags`: 页表项权限位集合。
- `sfence_vma`: RISC-V 刷新地址转换缓存的指令。

```
if(*pte & PTE_W)
    *pte = (*pte & ~PTE_W) | PTE_C;

pa = PTE2PA(*pte);
flags = PTE_FLAGS(*pte);
kaddrref((void *)pa);
if(mappages(new, i, PGSIZE, pa, flags) != 0){
    kfree((void *)pa);
    goto err;
}
sfence_vma();
```

答辩口径：COW fork 的关键不是复制内容，而是同时改父子页表：清可写位、加 COW 位、增加引用计数。这样父子读同一页没问题，写时再处理。

5.3 写故障复制与 copyout

设计思路：用户程序写 COW 页会触发 store page fault。内核检查该地址是否是合法 COW 页，如果是就分配新页、复制旧页内容、更新页表权限，然后让用户指令重新执行。copyout 也可能向用户页写数据，因此同样要处理 COW。

详细算法流程：

94. `usertrap` 中检查 `r_scause() == 15`，表示 store page fault，通常是写入只读页导致。
95. 用 `r_stval()` 取得触发故障的虚拟地址。
96. `cowalloc` 先 `PGROUNDDOWN` 对齐地址，再 walk 找页表项。
97. 确认该页 `PTE_V`、`PTE_U` 和 `PTE_C` 都存在，否则不是合法 COW 页，返回错误。
98. `kalloc` 分配新物理页，把旧物理页内容 `memmove` 到新页。
99. 把页表项改为新物理页地址，并设置 `PTE_W`、清除 `PTE_C`。
100. `kfree` 旧物理页，实际是否回收由引用计数决定。
101. `copyout` 在写用户页前先调用 `iscow` 和 `cowalloc`，保证内核向用户空间写入时也不会破坏共享页。

关键变量与英文名：

- store page fault: 写内存触发的缺页异常，RISC-V `scause` 值为 15。
- `r_stval`: 读取触发异常的虚拟地址。
- `cowalloc`: 本实现中处理 COW 页复制的函数。
- `iscow`: 判断某个虚拟地址是否映射到 COW 页。

```
} else if(r_scause() == 15){
    if(cowalloc(p->pagetable, r_stval()) < 0)
        p->killed = 1;
}

if(iscow(pagetable, va0) && cowalloc(pagetable, va0) < 0)
    return -1;
pa0 = walkaddr(pagetable, va0);
memmove((void *) (pa0 + (dstva - va0)), src, n);
```

答辩口径：答辩时可以按“页故障定位地址、验证 COW 标志、分配新页、复制数据、更新 PTE、减少旧页引用计数”来讲。

Lab 6 Multithreading (多线程)

本章的设计思路是从用户级线程切换扩展到真实 pthread 并发。uthread 关注寄存器上下文；ph 关注共享哈希表的锁粒度；barrier 关注条件变量和同步轮次。

6.1 uthread (用户级线程)

设计思路：用户级线程不进入内核调度器，而是在一个进程内部保存和恢复寄存器。设计上每个 thread 保存自己的栈和 callee-saved registers，调度器选择 RUNNABLE 线程后切换上下文。

详细算法流程：

102. 定义 struct thread_context 保存 ra、sp、s0-s11 等需要跨函数调用保留的寄存器。
103. 每个 struct thread 拥有独立 stack、state 和 context。
104. thread_init 把 all_thread[0] 作为当前主线程，状态设为 RUNNING。
105. thread_create 找到 FREE 线程槽，清空 context，把 ra 设置为线程函数地址，把 sp 设置到该线程栈顶并按 16 字节对齐。
106. thread_yield 把当前线程设为 RUNNABLE，然后调用 thread_schedule。
107. thread_schedule 从 current_thread 后面开始循环找 RUNNABLE 线程，采用简单 round-robin (轮转) 策略。
108. thread_switch 保存旧线程寄存器到 old context，再从新线程 context 恢复寄存器。

关键变量与英文名：

- thread: 线程，同一进程内可独立执行的控制流。
- context: 上下文，线程切换时必须保存的寄存器集合。
- callee-saved registers: 被调用函数必须保持不变的寄存器，例如 s0-s11。
- round-robin: 轮转调度，按固定顺序选择下一个可运行线程。

```
struct thread_context {
    uint64 ra;
    uint64 sp;
    uint64 s0;
    uint64 s1;
    ...
};

t->context.ra = (uint64)func;
t->context.sp = (uint64)(t->stack + STACK_SIZE);
t->context.sp -= t->context.sp % 16;
t->state = RUNNABLE;

thread_switch((uint64)&t->context,
              (uint64)&next_thread->context);
```

答辩口径：uthread 的核心不是创建内核线程，而是在用户态手动保存旧寄存器、恢复新寄存器。第一次运行新线程时，恢复出的 ra 就是线程函数入口。

6.2 ph_safe / ph_fast (哈希表并发)

设计思路：ph 的问题是多个 pthread 同时 put 会修改链表，导致节点丢失。最简单是全局锁，但会降低性能；本实现用 bucket-level lock (桶级锁)，每个哈希桶独立加锁。

详细算法流程：

109. 哈希表 table[NBUCKET] 按 key % NBUCKET 选择桶。
110. 给每个桶准备 pthread_mutex_t bucket_locks[NBUCKET]。
111. put(key, value) 先计算桶号 i，然后 pthread_mutex_lock(&bucket_locks[i])。
112. 在锁保护下遍历桶内链表，找到已有 key 就更新 value。
113. 如果没有找到，就分配新 entry 并插入到当前桶链表。

114. 修改完成后 `pthread_mutex_unlock(&bucket_locks[i])`。
115. `get` 主要是读操作，实验关注 `put` 阶段不丢 `key`；桶级锁能让不同桶的写入并行。

关键变量与英文名：

- `pthread`: POSIX thread, 宿主 Linux 下的线程库。
- `mutex`: mutual exclusion, 互斥锁, 保证同一时刻只有一个线程进入临界区。
- `bucket`: 哈希桶, 哈希表中保存一组链表节点的位置。
- `entry`: 哈希表链表节点, 保存 `key`、`value` 和 `next` 指针。

```
void put(int key, int value)
{
    int i = key % NBUCKET;
    pthread_mutex_lock(&bucket_locks[i]);

    struct entry *e = 0;
    for (e = table[i]; e != 0; e = e->next) {
        if (e->key == key)
            break;
    }
    if(e)
        e->value = value;
    else
        insert(key, value, &table[i], 0);

    pthread_mutex_unlock(&bucket_locks[i]);
}
```

答辩口径：答辩时说清楚锁粒度：全局锁正确但慢，桶锁仍然正确，同时允许不同 `bucket` 的 `put` 并发，所以 `ph_fast` 性能更好。

6.3 barrier（屏障同步）

设计思路：`barrier` 要保证一轮中所有线程都到达后才能进入下一轮。设计上用 `mutex` 保护计数，用 `condition variable` 让先到的线程睡眠，最后到达的线程广播唤醒。

详细算法流程：

116. `bstate.nthread` 记录当前轮已经到达 `barrier` 的线程数。
117. `bstate.round` 记录当前同步轮次，防止线程被错误唤醒后直接通过。
118. 线程进入 `barrier` 后先加锁，并保存 `my_round = bstate.round`。
119. 把 `bstate.nthread` 加 1；如果达到全局 `nthread`，说明本轮所有线程都到齐。
120. 最后到达的线程把计数清零、`round` 加 1，然后 `pthread_cond_broadcast` 唤醒所有等待线程。
121. 不是最后到达的线程进入 `while(my_round == bstate.round)` 循环，调用 `pthread_cond_wait` 睡眠并释放锁。
122. 被唤醒后重新检查 `round`，确认进入下一轮，再释放锁返回。

关键变量与英文名：

- `condition variable`: 条件变量，线程等待某个条件成立时使用。
- `broadcast`: 广播唤醒，唤醒所有等待在条件变量上的线程。
- `my_round`: 当前线程进入 `barrier` 时看到的轮次，用来判断是否真的进入下一轮。
- `spurious wakeup`: 虚假唤醒，所以条件变量等待通常写在 `while` 中。

```
pthread_mutex_lock(&bstate.barrier_mutex);
int my_round = bstate.round;
bstate.nthread++;

if(bstate.nthread == nthread){
    bstate.nthread = 0;
    bstate.round++;
    pthread_cond_broadcast(&bstate.barrier_cond);
} else {
```



```
while(my_round == bstate.round)
    pthread_cond_wait(&bstate.barrier_cond,
                     &bstate.barrier_mutex);
}
pthread_mutex_unlock(&bstate.barrier_mutex);
```

答辩口径：barrier 的答辩重点是 round。只用计数不够，因为被唤醒的线程需要知道自己等待的那一轮是否已经结束。

Lab 7 Networking（网络）

本章的设计思路是把网络包在三层之间传递：用户 socket 调用、协议栈 mbuf、E1000 网卡描述符环。发送路径从用户数据变成 mbuf，再交给 TX ring；接收路径从 RX ring 取 mbuf，再交给协议栈和 socket 队列。

7.1 e1000_transmit（网卡发送）

设计思路：E1000 使用 descriptor ring 管理发送缓冲区。软件找到 TDT 指向的空闲描述符，把 mbuf 地址和长度写进去，再推进 TDT 通知网卡。

详细算法流程：

123. 获取 e1000_lock，防止多个 CPU 同时修改发送环。
124. 读取 regs[E1000_TDT] 得到当前发送 tail 下标 idx。
125. 检查 tx_ring[idx].status 是否带有 E1000_TXD_STAT_DD，DD 表示网卡已经处理完这个描述符。
126. 如果描述符不空闲，释放锁并返回 -1，表示暂时无法发送。
127. 如果 tx_mbufs[idx] 保存着旧 mbuf，说明旧包已经发送完成，先 mbuffree。
128. 把新 mbuf 的 head 地址和 len 写入 tx_ring[idx]，设置 EOP 和 RS 命令位。
129. 清空 status，把 tx_mbufs[idx] 指向新 mbuf，最后把 TDT 推进到下一个环位置。

关键变量与英文名：

- E1000：实验模拟的 Intel 网卡设备。
- TX ring：transmit ring，发送描述符环。
- descriptor：描述符，告诉设备一块缓冲区的地址、长度和状态。
- TDT：transmit descriptor tail，发送环尾指针寄存器。

```
uint32 idx = regs[E1000_TDT];
if((tx_ring[idx].status & E1000_TXD_STAT_DD) == 0){
    release(&e1000_lock);
    return -1;
}
if(tx_mbufs[idx])
    mbuffree(tx_mbufs[idx]);

tx_ring[idx].addr = (uint64)m->head;
tx_ring[idx].length = m->len;
tx_ring[idx].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
tx_ring[idx].status = 0;
tx_mbufs[idx] = m;
regs[E1000_TDT] = (idx + 1) % TX_RING_SIZE;
```

答辩口径：发送函数的关键是 DD 位和 TDT。DD 保证当前槽位可复用，TDT 推进后网卡才知道有新包要发送。

7.2 e1000_recv（网卡接收）

设计思路：接收路径要不断查看 RDT 后面的描述符是否已经由设备写回。取走收到的 mbuf 后，必须立即补一个新的空 mbuf 给网卡继续接收。

详细算法流程：

130. 获取 e1000_lock，计算 $idx = (RDT + 1) \% RX_RING_SIZE$ 。

131. 检查 `rx_ring[idx].status` 是否有 `E1000_RXD_STAT_DD`；没有 `DD` 表示还没有新包。
132. 取出 `rx_mbufs[idx]` 作为收到的数据包 `m`。
133. 新分配 `replacement` mbuf，准备放回接收环给设备继续使用。
134. 根据描述符 `length` 设置 `m->len`，把 `replacement` 的 `head` 地址写回 `rx_ring[idx].addr`。
135. 清空描述符 `status`，把 `regs[E1000_RDT]` 更新到 `idx`，表示这个槽位已经重新交给设备。
136. 释放网卡锁后调用 `net_rx(m)`，避免协议栈处理时长时间占用设备锁。
137. 处理完一个包后重新加锁继续检查，直到没有 `DD` 的描述符。

关键变量与英文名：

- `RX ring`: receive ring，接收描述符环。
- `RDT`: receive descriptor tail，软件已经交还给设备的接收环位置。
- `mbuf`: message buffer，网络包缓冲结构，保存 `head` 指针和 `len` 长度。
- `net_rx`: 协议栈入口，负责解析以太网、IP、UDP 等头部。

```
while(1){
    uint32 idx = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
    if((rx_ring[idx].status & E1000_RXD_STAT_DD) == 0)
        break;

    struct mbuf *m = rx_mbufs[idx];
    struct mbuf *replacement = mbufalloc(0);
    m->len = rx_ring[idx].length;
    rx_mbufs[idx] = replacement;
    rx_ring[idx].addr = (uint64)replacement->head;
    rx_ring[idx].status = 0;
    regs[E1000_RDT] = idx;

    release(&e1000_lock);
    net_rx(m);
    acquire(&e1000_lock);
}
```

答辩口径：接收函数最容易被问的是为什么要 `replacement`。因为原 `mbuf` 已经交给协议栈，网卡环上必须补一个新的空缓冲区，否则后续包没有地方写。

7.3 socket/UDP 调用链

设计思路：用户程序通过 `socket` 文件描述符读写，内核用 `mbuf` 队列连接网络中断和用户 `read`。发送时 `copyin` 到 `mbuf`，接收时中断把 `mbuf` 入队并唤醒等待者。

详细算法流程：

138. `sockwrite` 从用户地址 `copyin` 数据到新 `mbuf` 的 `payload` 区域。
139. 调用 `net_tx_udp` 添加 UDP、IP、Ethernet 头部，最终进入 `e1000_transmit`。
140. 接收时 `E1000` 中断调用 `e1000_recv`，协议栈 `net_rx` 解析 Ethernet/IP/UDP。
141. `net_rx_udp` 提取源 IP、源端口和目的端口，调用 `sockrecvudp`。
142. `sockrecvudp` 在线性 `socket` 表中找到匹配 `raddr`、`lport`、`rport` 的 `socket`。
143. 找到后把 `mbuf` 放入 `socket` 的 `rxq` 队列，并 `wakeup(&si->rxq)` 唤醒正在 `sockread` 的进程。
144. `sockread` 若队列为空就 `sleep`，醒来后取出 `mbuf`，用 `copyout` 写回用户缓冲区。

关键变量与英文名：

- `UDP`: user datagram protocol，用户数据报协议，提供无连接报文传输。
- `socket`: 网络通信端点，在 `xv6` 中也通过 `file` 结构接入文件描述符表。
- `rxq`: receive queue，接收队列，保存已经收到但用户尚未读取的 `mbuf`。
- `copyin/copyout`: 内核和用户地址空间之间复制数据的安全函数。

```
if(copyin(pr->pagetable, mbufput(m, n), addr, n) == -1) {
    mbuffree(m);
    return -1;
}
```

```

}
net_tx_udp(m, si->raddr, si->lport, si->rport);

mbufq_pushtail(&si->rxq, m);
wakeup(&si->rxq);

m = mbufq_pophead(&si->rxq);
copyout(pr->pagetable, addr, m->head, len);

```

答辩口径：可以把网络 Lab 讲成两条链：发送链是 user -> socket -> mbuf -> UDP/IP/Ethernet -> E1000；接收链是 E1000 interrupt -> mbuf -> protocol stack -> socket rxq -> user。

Lab 8 Lock（锁优化）

本章的设计思路是减少共享热点，而不是取消锁。kalloc 原来只有一个全局 freelist，所有 CPU 都抢同一把锁；bcache 原来只有一个全局链表，所有磁盘块查找和替换也抢同一把锁。优化方式是拆分共享数据结构。

8.1 kalloc per-CPU freelist（每 CPU 空闲链表）

设计思路：把一个全局 kmem 拆成 kmem[NCPU]，每个 CPU 优先从自己的 freelist 分配和释放。只有本地没有空闲页时，才从其他 CPU 偷取一批页，减少常规路径的锁竞争。

详细算法流程：

145. 把 kmem 改成数组，每个元素有自己的 spinlock 和 freelist。
146. kfree 时通过 push_off 关闭中断，再 cpuid 获取当前 CPU 编号，避免进程迁移导致选错本地链表。
147. 把释放的页插入 kmem[id].freelist，然后释放锁并 pop_off。
148. kalloc 也先关闭中断并读取 id，优先尝试当前 CPU 的 freelist。
149. 如果本地 freelist 为空，就依次检查其他 CPU 的 freelist。
150. 从其他 CPU 偷取时一次取最多一批页，把第一张作为返回值，剩余页挂回本地 freelist。
151. 这样大多数分配释放只竞争本 CPU 的锁，只有缺页时才跨 CPU 访问。

关键变量与英文名：

- per-CPU：每个 CPU 各自拥有一份数据结构，减少共享竞争。
- push_off/pop_off：成对关闭和恢复中断，保证当前代码段不会被中断并迁移到其他 CPU。
- steal：从其他 CPU 的 freelist 偷取空闲页。
- spinlock：自旋锁，获取不到时循环等待，适合短临界区。

```

push_off();
id = cpuid();
acquire(&kmem[id].lock);
r = kmem[id].freelist;
if(r)
    kmem[id].freelist = r->next;
release(&kmem[id].lock);

if(r == 0){
    for(int i = 0; i < NCPU; i++){
        if(i == id) continue;
        acquire(&kmem[i].lock);
        r = kmem[i].freelist;
        ...
        release(&kmem[i].lock);
        if(r) break;
    }
}
pop_off();

```

答辩口径：答辩时说清楚：per-CPU 并不是完全不共享，而是让高频路径本地化；偶尔 steal 保证各 CPU 空闲页不均衡时仍能工作。

8.2 bcache bucket locks（缓冲区缓存哈希桶锁）

设计思路：buffer cache 的访问按 dev 和 blockno 哈希到不同 bucket。查找命中时只锁对应 bucket；替换未命中时用 evict_lock 串行化全局替换，避免同一个磁盘块被并发放进两个桶。

详细算法流程：

152. 定义 NBUCKET 个 bucket，每个 bucket 有独立 spinlock 和双向链表 head。
153. bhash(dev, blockno) 计算磁盘块属于哪个桶。
154. bget 先锁目标 bucket，在链表中查找是否已有相同 dev/blockno 的 buf。
155. 命中时 refcnt++，释放 bucket 锁，然后获取该 buf 的 sleeplock 返回。
156. 未命中时释放 bucket 锁，获取 evict_lock，防止多个 CPU 同时做全局替换。
157. 持有 evict_lock 后重新检查目标 bucket，避免刚才释放锁期间其他 CPU 已插入该块。
158. 仍未命中，就从任意 bucket 找 refcnt == 0 的 buf，摘出后插入目标 bucket。
159. brelse 释放 buf sleeplock 后，回到对应 bucket 减 refcnt，为 0 时移动到 MRU 位置。

关键变量与英文名：

- bcache：buffer cache，磁盘块缓冲区缓存。
- buf：缓存块结构，保存 dev、blockno、valid、refcnt 和 data。
- refcnt：引用计数，表示当前有多少使用者持有该 buf。
- MRU：most recently used，最近使用位置。

```
h = bhash(dev, blockno);
bk = &bcache.bucket[h];
acquire(&bk->lock);
for(b = bk->head.next; b != &bk->head; b = b->next){
    if(b->dev == dev && b->blockno == blockno){
        b->refcnt++;
        release(&bk->lock);
        acquiresleep(&b->lock);
        return b;
    }
}
release(&bk->lock);

acquire(&bcache.evict_lock);
// recheck, then recycle an unused buffer
```

答辩口径：bcache 的设计口径是“命中路径按桶并发，替换路径全局串行”。这样既降低常规读写竞争，又避免重复缓存同一块。

8.3 锁竞争统计

设计思路：为了证明优化有效，实验统计每把锁 acquire 次数和 test-and-set 失败次数。优化目标不是锁数量越少越好，而是热点锁的 #test-and-set 显著下降。

详细算法流程：

160. 在 spinlock 结构中维护 n 和 nts，n 记录 acquire 调用次数，nts 记录自旋等待次数。
161. acquire 进入时用原子加法增加 n。
162. 每次 __sync_lock_test_and_set 失败并继续自旋时，增加 nts。
163. statslock 遍历锁表，只输出 kmem 和 bcache 相关锁，以及竞争最严重的前几把锁。
164. 运行 kallocetest 和 bcachetest 后观察 tot 是否明显下降，判断拆锁是否有效。

关键变量与英文名：

- test-and-set：原子交换指令，用于尝试把锁从未持有改为持有。

- nts: number of test-and-set failures, 自旋失败次数。
- nacquire: 锁获取次数, 表示锁被使用的频率。
- contention: 锁竞争, 多个 CPU 同时尝试获取同一把锁。

```
while(__sync_lock_test_and_set(&lk->locked, 1) != 0) {
#ifdef LAB_LOCK
    __sync_fetch_and_add(&(lk->nts), 1);
#endif
}

n = snprintf(buf, sz,
    "lock: %s: #test-and-set %d #acquire() %d\n",
    lk->name, lk->nts, lk->n);
```

答辩口径: 答辩时把指标解释清楚: acquire 多不一定坏, test-and-set 失败多才说明等待严重。优化后热点锁等待次数下降, 说明拆锁有效。

Lab 9 File system (文件系统)

本章的设计思路是扩展 xv6 文件系统能力。bigfile 通过 double-indirect block (双重间接块) 扩大单文件容量; symlink 通过新 inode 类型保存目标路径, 并在 open 中按需解析。

9.1 bigfile (大文件与双重间接块)

设计思路: xv6 原文件只支持直接块和一级间接块, 容量有限。为加入双重间接块, 需要牺牲一个直接块, 把 addrs 最后一项作为二级索引入口。

详细算法流程:

165. 把 NDIRECT 从 12 改为 11, 预留 addrs[NDIRECT+1] 给双重间接块。
166. 定义 NINDIRECT = BSIZE / sizeof(uint), 表示一个间接块可保存多少个块号。
167. 定义 NDINDIRECT = NINDIRECT * NINDIRECT, 表示双重间接块可索引的数据块数。
168. bmap 中先处理 bn < NDIRECT 的直接块。
169. 再处理 bn < NINDIRECT 的一级间接块, 逻辑与原 xv6 一致。
170. 剩余 bn 进入双重间接区域, 先算 idx1 = bn / NINDIRECT, idx2 = bn % NINDIRECT。
171. 如果双重间接块不存在, 先 balloc 分配; 再读取第一级索引块。
172. 如果第一级索引项为空, 分配一个普通间接块; 再读这个间接块并根据 idx2 分配或返回数据块。
173. itrunc 释放文件时必须反向释放: 先释放所有二级数据块, 再释放一级间接块, 最后释放双重间接块本身。

关键变量与英文名:

- inode: 索引节点, 记录文件类型、大小和数据块地址。
- direct block: 直接块, inode 中直接保存的数据块号。
- indirect block: 间接块, 块里保存一组数据块号。
- double-indirect block: 双重间接块, 先索引间接块, 再由间接块索引数据块。

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDINDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDINDIRECT)

if(bn < NDINDIRECT){
    uint idx1 = bn / NINDIRECT;
    uint idx2 = bn % NINDIRECT;
    if((addr = ip->addrs[NDIRECT+1]) == 0)
        ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[idx1]) == 0){
        a[idx1] = addr = balloc(ip->dev);
        log_write(bp);
    }
}
```

```

    }
    brelse(bp);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[idx2]) == 0){
        a[idx2] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    return addr;
}

```

答辩口径：答辩时画两级数组最清楚：inode 指向双重间接块，双重间接块指向多个一级间接块，每个一级间接块再指向数据块。

9.2 symlink（符号链接）

设计思路：符号链接本质是一个特殊 inode，文件内容保存目标路径字符串。open 默认跟随符号链接，除非用户传入 O_NOFOLLOW。为避免循环链接，需要限制解析深度。

详细算法流程：

174. 在 stat.h 增加 T_SYMLINK 文件类型，在 fcntl.h 增加 O_NOFOLLOW 打开标志。
175. 新增 sys_symlink(target, path)，先 create(path, T_SYMLINK, 0, 0) 创建符号链接 inode。
176. 把 target 路径字符串通过 writei 写入该 inode 的文件内容中。
177. sys_open 普通打开路径后，如果 ip->type == T_SYMLINK 且没有 O_NOFOLLOW，就进入循环解析。
178. 每次解析先 readi 读出链接保存的目标路径，再 iunlockput 当前链接 inode。
179. 用 namei(path) 查找目标路径对应 inode，并继续检查它是否仍是符号链接。
180. 用 depth 限制最大解析次数，例如超过 10 次返回错误，防止 a -> b -> a 这种循环。
181. 如果设置 O_NOFOLLOW，则 open 返回符号链接本身，而不是目标文件。

关键变量与英文名：

- symlink: symbolic link, 符号链接，保存另一个路径名的特殊文件。
- O_NOFOLLOW: 打开时不跟随符号链接的标志。
- namei: 根据路径名查找 inode 的函数。
- depth: 符号链接递归解析深度，用来防止循环。

```

while(ip->type == T_SYMLINK && (omode & O_NOFOLLOW) == 0){
    if(++depth > 10){
        iunlockput(ip);
        end_op();
        return -1;
    }
    if(readi(ip, 0, (uint64)path, 0, MAXPATH) <= 0){
        iunlockput(ip);
        end_op();
        return -1;
    }
    path[MAXPATH-1] = 0;
    iunlockput(ip);
    if((ip = namei(path)) == 0){
        end_op();
        return -1;
    }
    ilock(ip);
}

```

答辩口径：symlink 的核心是“路径转发”而不是硬链接。硬链接指向同一个 inode，符号链接保存目标路径，因此目标可以不存在，也可能形成循环。

Lab 10 Mmap (memory map, 内存映射)

本章的设计思路是用 VMA 描述一段文件映射区，但暂时不分配物理页。真正访问映射地址时触发 page fault，内核再从文件读取一页内容建立映射。munmap 和 exit 负责写回与回收。

10.1 VMA 与 sys_mmap

设计思路：mmap 不立即读取整个文件，而是记录映射关系。每个进程维护 VMA 数组，描述映射起始地址、长度、权限、共享方式、文件和偏移。

详细算法流程：

182. 在 proc.h 中定义 struct vma，字段包括 used、addr、len、prot、flags、offset 和 file。
183. sys_mmap 读取 addr、length、prot、flags、fd、offset 六个参数；实验中 addr 参数可以忽略，由内核选择地址。
184. 检查 length > 0，offset 按页对齐，flags 只能是 MAP_SHARED 或 MAP_PRIVATE。
185. 检查 prot 权限是否合法；如果请求读权限，文件必须 readable；如果 MAP_SHARED + 写权限，文件必须 writable。
186. 在 p->vma[NVMA] 中找一个 unused 槽位。
187. len = PGROUNDUP(length)，把映射长度向上取整到页大小。
188. alloc_mmap_addr 从 TRAPFRAME 下方向低地址寻找空闲区域，避免和进程已有 heap 冲突。
189. 填写 VMA 字段，并 filedup(f) 增加文件引用计数，最后返回映射起始地址。

关键变量与英文名：

- VMA: virtual memory area，虚拟内存区域，用来描述一段连续映射。
- PROT_READ/PROT_WRITE/PROT_EXEC：映射页允许读、写、执行的权限。
- MAP_SHARED：共享映射，修改需要写回文件。
- MAP_PRIVATE：私有映射，修改不写回原文件。

```
struct vma {
    int used;
    uint64 addr;
    uint64 len;
    int prot;
    int flags;
    uint64 offset;
    struct file *file;
};

len = PGROUNDUP((uint64)length);
addr = alloc_mmap_addr(p, len);
v->used = 1;
v->addr = addr;
v->len = len;
v->prot = prot;
v->flags = flags;
v->offset = offset;
v->file = filedup(f);
return addr;
```

答辩口径：mmap 的设计口径是“先登记 VMA，后按需分配物理页”。这样大文件映射不会一开始就占用大量内存。

10.2 mmapfault (映射区缺页处理)

设计思路：访问 VMA 中尚未分配的页会触发 page fault。内核根据 fault address 找到 VMA，检查权限，从文件读取一页，再建立页表映射。

详细算法流程：

190. usertrap 检查 scause 是否为 13、15 或 12，分别表示 load、store、instruction page fault。

191. 调用 `mmapfault(r_stval(), r_scause())`, `r_stval` 提供触发异常的虚拟地址。
192. `mmapfault` 先 `PGROUNDDOWN` 对齐地址, 调用 `find_vma` 查找包含该地址的 VMA。
193. 根据 `scause` 检查访问权限: 读故障必须有 `PROT_READ`, 写故障必须有 `PROT_WRITE`, 执行故障必须有 `PROT_EXEC`。
194. 如果该虚拟页已经存在有效 PTE, 说明不是合法延迟分配, 返回错误。
195. `kalloc` 分配物理页并清零, 计算文件偏移 `off = v->offset + (a - v->addr)`。
196. `ilock` 文件 inode, 调用 `readi` 从文件偏移处读 `PGSIZE` 字节到新物理页。
197. 根据 `prot` 设置 `PTE_R`、`PTE_W`、`PTE_X` 权限, 调用 `mappages` 建立用户页映射。
198. 返回 `usertrapret` 后, 触发缺页的用户指令会重新执行, 这次地址已经有效。

关键变量与英文名:

- `page fault`: 缺页异常, 访问页表中不存在或权限不允许的页面时触发。
- `scause 13`: `load page fault`, 读缺页。
- `scause 15`: `store page fault`, 写缺页。
- `scause 12`: `instruction page fault`, 取指缺页。

```

} else if(r_scause() == 13 || r_scause() == 15 || r_scause() == 12){
    intr_on();
    if(mmapfault(r_stval(), r_scause()) < 0)
        p->killed = 1;
}

a = PGROUNDDOWN(va);
v = find_vma(p, a);
if(scause == 13 && (v->prot & PROT_READ) == 0)
    return -1;
if(scause == 15 && (v->prot & PROT_WRITE) == 0)
    return -1;
mem = kalloc();
off = v->offset + (a - v->addr);
readi(v->file->ip, 0, (uint64)mem, (uint)off, PGSIZE);
mappages(p->pagetable, a, PGSIZE, (uint64)mem, perm);

```

答辩口径: 答辩时把 `mmapfault` 讲成“缺页即加载”。VMA 记录文件和权限, `fault address` 决定读文件的哪一页, 页表映射建立后原指令继续执行。

10.3 munmap、fork 和退出回收

设计思路: `munmap` 负责取消映射。对于 `MAP_SHARED` 且可写的已映射页, 需要先写回文件; 对于未实际访问过的页, 页表里没有 PTE, 可以只更新 VMA。fork 和 `exit` 要维护文件引用计数。

详细算法流程:

199. `sys_munmap` 读取 `addr` 和 `length`, 调用 `munmap_range`。
200. `munmap_range` 检查 `addr` 页对齐、`len` 向上取整, 并确认 `[addr, end)` 完全落在某个 VMA 内。
201. 实验只允许从 VMA 开头或结尾取消映射, 不支持从中间挖空。
202. 遍历被取消的每一页, `walk` 查找 PTE; 如果页从未被访问, PTE 可能不存在, 直接跳过。
203. 如果 PTE 有效且 VMA 是 `MAP_SHARED` 并带 `PROT_WRITE`, 调用 `mmap_writeback` 写回文件。
204. 调用 `uvmunmap` 取消页表映射并释放物理页。
205. 如果整个 VMA 都被取消, 清空 VMA 并 `fileclose`; 如果只取消开头或结尾, 则调整 `addr`、`offset`、`len`。
206. `fork` 复制父进程 VMA 数组, 并对每个映射文件 `filedup`。
207. `exit` 调用 `munmap_all`, 把所有仍存在的映射写回、释放并关闭文件引用。

关键变量与英文名:

- `munmap`: 取消一段 `mmap` 映射。
- `writeback`: 写回, 把内存中修改过的数据写回文件。

- `filedup`: 增加 `struct file` 的引用计数。
- `fileclose`: 减少文件引用计数, 计数为 0 时真正关闭。

```
for(uint64 a = addr; a < end; a += PGSIZE){
    pte_t *pte = walk(p->pagetable, a, 0);
    if(pte && (*pte & PTE_V)){
        if((v->flags & MAP_SHARED) && (v->prot & PROT_WRITE)){
            if(mmap_writeback(v, a, PTE2PA(*pte)) < 0)
                return -1;
        }
        uvmunmap(p->pagetable, a, 1, 1);
    }
}

if(addr == v->addr && len == v->len){
    struct file *f = v->file;
    memset(v, 0, sizeof(*v));
    fileclose(f);
}
```

答辩口径: 答辩时强调 `mmap` 的生命周期: `mmap` 创建 VMA, `page fault` 填充物理页, `munmap` 写回并释放, `fork` 复制 VMA 与文件引用, `exit` 统一清理。

A 常见英文变量与操作系统概念速查

这一部分用于快问快答复习。答辩时如果听到英文变量名, 可以先判断它属于进程、页表、陷入、文件系统、网络还是并发同步。

- `proc`: process, 进程结构, `xv6` 用 `struct proc` 保存进程状态、页表、`trapframe`、打开文件等。
- `trapframe`: 陷入帧, 保存用户态寄存器, 系统调用、中断、异常返回用户态都依赖它。
- `pagetable`: 页表根指针, 指向三级页表的根页。
- `PTE`: page table entry, 页表项, 保存物理页号和权限位。
- `walk`: 页表查找函数, 沿 `Sv39` 三级页表找到某虚拟地址对应的 `PTE`。
- `copyin/copyout`: 用户空间与内核空间之间复制数据的安全接口。
- `inode`: 文件系统索引节点, 保存文件类型、大小和数据块地址。
- `buf`: buffer cache 中的磁盘块缓存对象。
- `mbuf`: 网络包缓冲, 保存网络报文内容和长度。
- `lock`: 锁, 保护共享数据, 避免并发修改造成竞争。
- `refcnt`: reference count, 引用计数, 用于判断资源是否还能释放。
- `VMA`: virtual memory area, `mmap` 用来描述一段文件映射区域。

B 答辩时的通用说明模板

208. 先说目标: 这个子实验要补齐 `xv6` 的哪一种能力, 例如系统调用、页表访问、缺页处理或锁优化。
209. 再说状态: 新增或修改了哪个结构体字段, 例如 `tracemask`、`alarmframe`、`PTE_C`、`vma`。
210. 再说触发路径: 用户命令、系统调用、`trap`、设备中断或文件读写会进入哪段函数。
211. 再说核心算法: 按照数据流讲, 不要只背函数名。比如 `COW` 是页表共享到写故障复制, `mmap` 是 VMA 登记到缺页加载。
212. 最后说边界: 为什么要检查参数、权限、引用计数、路径循环、页边界或锁顺序。